

AIDLC Control Platform — Complete Project Blueprint

0. Executive Summary

Dự án cần triển khai một **AIDLC Control Platform** — một sản phẩm chạy được thật, mô phỏng và tự động hóa luồng phát triển phần mềm end-to-end bằng team AI agents gồm **PO Agent, UI/UX Agent, DEV Agent và QA Agent**.

Output cuối cùng không phải là diagram, report hay prompt rời rạc. Output phải là một **web app product-ready**, trong đó user có thể nhập feature request, hệ thống chạy qua các phase AIDLC, tạo artifact ở từng phase, cho phép human approve/reject/request changes tại quality gates, lưu audit trail, và tạo final review packet để quyết định release.

Data và agent output có thể mock, nhưng workflow, UI, backend, state transition, approval, artifact management và audit trail phải chạy thật, mượt mà và có thể demo được.

1. Đề bài

1.1 Đề bài gốc

Tự động hóa toàn bộ luồng phát triển phần mềm end-to-end bằng team agent, mỗi agent một việc. Hệ thống phối hợp giữa Agent PO, Agent UI/UX, Agent Developer, Agent QA thành một vòng SDLC. Có hỗ trợ kiểm soát các agent để đảm bảo đúng mục tiêu. Demo chạy thử trên một dự án phát triển phần mềm.

1.2 Diễn giải đề bài theo hướng triển khai

Hệ thống cần chứng minh 4 ý chính:

1. Có workflow phát triển phần mềm end-to-end.
2. Có nhiều agent theo vai trò SDLC: PO, UI/UX, DEV, QA.
3. Các agent phối hợp với nhau thông qua artifact handoff.
4. Human tham gia vào các quality gates để kiểm soát chất lượng.

1.3 Expectation của mentor

Mentor đang kỳ vọng team tập trung vào:

- Triển khai AIDLC thật, không chỉ mô tả lý thuyết.
- Human tham gia ở những điểm nào để kiểm soát chất lượng.
- Output cuối phải đúng đề bài và chất lượng tốt.
- Product phải chạy tốt, mượt mà.
- Data có thể mock nhưng trải nghiệm phải giống sản phẩm thật.

2. Problem Statement

Hiện nay nhiều AI coding tools có thể hỗ trợ viết code nhanh hơn, nhưng phần lớn chỉ tập trung vào một tác vụ riêng lẻ như code generation, code completion hoặc bug fixing. Chúng chưa thể hiện rõ một quy trình phát triển phần mềm end-to-end có phân vai, có artifact trung gian, có kiểm soát chất lượng, có human review và có traceability từ requirement đến QA.

Dự án này giải quyết vấn đề đó bằng cách xây dựng một **AIDLC Control Platform**, nơi các agent đóng vai trò như một team phát triển phần mềm gồm PO, UI/UX, DEV và QA. Mỗi agent thực hiện một phase trong SDLC và tạo artifact rõ ràng. Human tham gia tại các quality gates để approve, reject hoặc request changes nhằm đảm bảo output đúng yêu cầu và đạt chất lượng tốt.

3. Product Vision

3.1 Product name

End-to-End Autonomous Software Factory (Multi-AI Agent Team)

3.2 One-line positioning

A human-governed multi-agent platform that automates the software development lifecycle from feature request to QA-ready release candidate.

3.3 Vietnamese positioning

Nền tảng điều phối quy trình phát triển phần mềm bằng team AI agents có human quality control.

3.4 Final product goal

User có thể thực hiện một demo end-to-end như sau:

1. Nhập feature request.
2. PO Agent tạo PRD, user stories và acceptance criteria.
3. Human review và approve/reject requirement.
4. UI/UX Agent tạo user flow, screen spec và wireframe spec.
5. Human review và approve/reject UX.
6. DEV Agent tạo implementation plan, mock code diff và risk assessment.
7. QA Agent tạo test cases, QA report và acceptance criteria coverage.
8. Human review QA result và final review packet.
9. Human quyết định release / hold / request changes.
10. Audit trail lưu toàn bộ quá trình.

4. How BMAD, OMO and HITL Are Used

Đây là phần quan trọng nhất để tránh hiểu nhầm rằng team chỉ copy tool có sẵn.

4.1 BMAD được dùng như thế nào?

BMAD được dùng làm **reference model cho AIDLC artifact workflow**.

BMAD có điểm mạnh là chia quy trình phát triển phần mềm thành các phase và role rõ ràng như Business Analyst, Product Manager, UX Designer, Software Engineer. Mỗi phase tạo artifact để phase sau sử dụng.

BMAD mapping vào hệ thống

BMAD concept	Product implementation
Mary / Business Analyst	PO Agent hoặc Requirement Analysis phase
John / Product Manager	PO Agent tạo PRD + user stories + acceptance criteria
Sally / UX Designer	UI/UX Agent tạo UX spec + user flow + wireframe spec
Amelia / Engineer	DEV Agent tạo implementation plan + code diff
BMAD artifact files	Artifact Store trong database
User chuyển session	Human approval gate
Readiness check	Quality Gate trước khi DEV/QA chạy

BMAD-inspired artifacts

PO Agent outputs:

- prd.md
- user_stories.md
- acceptance_criteria.md
- scope.md

UI/UX Agent outputs:

- ux_spec.md
- user_flow.md
- wireframe_spec.md
- component_inventory.md

DEV Agent outputs:

- implementation_plan.md
- mock_code_diff.md
- changed_files.json

- risk_assessment.md

QA Agent outputs:

- test_cases.md
- qa_report.md
- ac_coverage_matrix.md

Final output:

- final_review_packet.md
- audit_trail.json

4.2 OMO được dùng như thế nào?

Oh-My-OpenAgent được dùng làm **reference model cho orchestration engine**.

OMO có điểm mạnh là điều phối agent execution thông qua concepts như IntentGate, Sisyphus Orchestrator, Category Router, lifecycle hooks, permissions, recovery và self-refinement loop.

Trong dự án này, team không cần implement full OMO phức tạp. Thay vào đó, team lấy các pattern cốt lõi và biến thành backend workflow engine.

OMO mapping vào hệ thống

OMO concept	Product implementation
ultrawork [task]	User nhập feature request
IntentGate	Requirement Intake / Intent Classification
Sisyphus Orchestrator	Workflow Orchestrator service
Category Router	Route task đến PO / UX / DEV / QA Agent
tmux parallel execution	UI timeline / agent run status
Ralph Loop	Agent self-review trước khi gửi human review
Todo Enforcer	Workflow completeness checker
Lifecycle hooks	before/after phase hooks, audit hooks, approval hooks
Config permissions	Risk-based action policy
Session recovery	Saved workflow state / resume support

OMO-inspired backend flow

Feature Request

- IntentGate validates task clarity
- Orchestrator creates workflow run
- Router selects next agent phase
- Agent Runner executes selected agent
- Lifecycle hook logs event

- Self-review validates artifact
- Human gate approves/rejects
- Todo Enforcer checks missing phase/artifact
- Workflow proceeds or loops back

4.3 HITL được dùng như thế nào?

HITL được dùng làm **quality control layer**.

Team không implement HITL theo kiểu approve mọi hành động, vì như vậy sẽ gây approve fatigue. Thay vào đó:

Observe everything.
Approve only quality gates.

Human có thể quan sát toàn bộ quá trình qua audit trail và workflow board, nhưng chỉ cần approve tại các checkpoint quan trọng.

Human quality gates

Gate	Phase	Human kiểm soát gì?
Gate 1	Sau PO Agent	Requirement đúng đề bài chưa? AC rõ chưa?
Gate 2	Sau UI/UX Agent	UX flow hợp lý chưa? Đủ states chưa?
Gate 3	Sau DEV Agent	Implementation plan/code diff có risk không?
Gate 4	Sau QA Agent	Test có cover đủ AC không? Có fail/blocker không?
Gate 5	Final Review	Release / hold / rework?

5. Core Workflow

5.1 End-to-end workflow

```
flowchart TD
    A[User creates feature request] --> B[IntentGate]
    B --> C{Clear enough?}
    C -->|No| C1[Ask human clarification]
    C1 --> B
    C -->|Yes| D[Workflow Orchestrator]
    D --> E[PO Agent]
```

```

Generate PRD + Stories + AC]
  E --> E1[PO Self-review
Check output contract]
  E1 --> G1{Human Gate 1
Requirement approval}
  G1 -->|Reject / Request changes| E
  G1 -->|Approve| F[UI/UX Agent
Generate UX spec + flow + wireframe]

  F --> F1[UX Self-review
Check output contract]
  F1 --> G2{Human Gate 2
UX approval}
  G2 -->|Reject / Request changes| F
  G2 -->|Approve| H[DEV Agent
Generate implementation plan + mock diff]

  H --> H1[Risk assessment
Auth / DB / security / dependency?]
  H1 --> G3{Human Gate 3
High-risk approval?}
  G3 -->|Reject / Request safer plan| H
  G3 -->|Approve / Low risk| I[QA Agent
Generate test cases + QA report]

  I --> I1[QA validates AC coverage]
  I1 --> G4{Human Gate 4
QA review}
  G4 -->|Fail / Request fix| H
  G4 -->|Pass| J[Final Review Packet]

  J --> G5{Human Gate 5
Release decision}
  G5 -->|Release| K[Release Candidate Ready]
  G5 -->|Hold| L[Hold / New Sprint]

```

5.2 Workflow states

```

DRAFT
→ INTAKE_REVIEW
→ PO_RUNNING
→ PO_REVIEW
→ UX_RUNNING
→ UX_REVIEW
→ DEV_RUNNING
→ DEV_REVIEW
→ QA_RUNNING
→ QA_REVIEW
→ FINAL_REVIEW
→ READY / HOLD

```

Reject/rework states:

PO_REWORK
UX_REWORK
DEV_REWORK
QA_FAILED

6. Pre-Testcase Foundation: Data Contract → Agent Contract → Test Case Contract

Đây là phần phải xử lý trước khi viết test cases. Nếu không có contract, QA Agent sẽ sinh test cases cảm tính và khó chứng minh đúng đề bài.

6.1 Core chain

Feature Request
→ Data Contract
→ Agent Contract
→ Output Contract
→ Acceptance Criteria
→ Test Case Contract
→ Test Cases
→ QA Report

Không nên đi thẳng:

Feature Request → Test Cases

Vì như vậy thiếu chuẩn dữ liệu, thiếu chuẩn output và thiếu oracle để đánh giá đúng/sai.

7. Data Contract Layer

7.1 Purpose

Data Contract định nghĩa dữ liệu nào được xử lý trong hệ thống, format của dữ liệu, agent nào đọc/ghi dữ liệu đó, và dữ liệu đó phục vụ phase nào trong AIDLC.

7.2 Required data groups

Data Group	Purpose	Producer	Consumer
Feature Request Data	Input ban đầu của user	User	IntentGate, PO Agent
Project Context Data	Bối cảnh project	User/System	All agents

Data Group	Purpose	Producer	Consumer
Requirement Data	Requirement chuẩn hóa	PO Agent	UX, DEV, QA Agent
UX Data	Thiết kế trải nghiệm	UI/UX Agent	DEV, QA Agent
Implementation Data	Plan/code diff	DEV Agent	QA Agent, Human reviewer
QA Data	Test cases/report	QA Agent	Human reviewer
HITL Data	Human decisions	Human	Workflow Engine
Audit Data	Trace toàn bộ workflow	System	Human/Mentor

7.3 Feature Request schema

```
{
  "id": "FR-001",
  "title": "Google Login",
  "description": "Add Google login for task management app",
  "priority": "High",
  "target_user": "End user",
  "business_goal": "Reduce login friction",
  "constraints": [
    "Must support error state",
    "Must not break existing login"
  ],
  "created_by": "human_user",
  "status": "DRAFT"
}
```

7.4 Project Context schema

```
{
  "project_id": "PRJ-001",
  "project_name": "Task Management App",
  "tech_stack": {
    "frontend": "React",
    "backend": "FastAPI",
    "database": "SQLite"
  },
  "existing_features": [
    "Email login",
    "Task dashboard",
    "User profile"
  ],
  "constraints": [
    "Do not break existing login",
    "Keep UI consistent with current design",
    "Mock data is allowed for demo"
  ]
}
```



```
]
}
```

7.5 HITL Decision schema

```
{
  "id": "APR-001",
  "workflow_run_id": "WR-001",
  "gate": "REQUIREMENT_GATE",
  "decision": "APPROVE",
  "comment": "Acceptance criteria are clear enough.",
  "reviewer": "human_user",
  "created_at": "2026-05-26T10:00:00Z"
}
```

7.6 Audit Log schema

```
{
  "id": "LOG-001",
  "workflow_run_id": "WR-001",
  "actor": "PO_AGENT",
  "action": "GENERATE_PRD",
  "artifact_id": "ART-001",
  "status": "SUCCESS",
  "timestamp": "2026-05-26T10:01:00Z"
}
```

8. Agent Contract Layer

8.1 Purpose

Agent Contract định nghĩa rõ mỗi agent:

- Nhận input gì?
- Được phép làm gì?
- Phải tạo output gì?
- Output hợp lệ phải thỏa rules nào?
- Khi nào cần human review?

8.2 PO Agent Contract

Input

Feature Request Data
Project Context Data

Responsibilities

- Làm rõ problem statement.
- Xác định target user.
- Tạo user stories.
- Tạo acceptance criteria.
- Xác định scope/out-of-scope.

Output

prd.md
user_stories.md
acceptance_criteria.md
scope.md

Output quality rules

- Mỗi user story phải có ít nhất 1 acceptance criterion.
- Acceptance criteria phải measurable.
- Scope và out-of-scope phải rõ.
- Constraint không được mâu thuẫn với project context.

Human gate

Gate 1: Requirement Approval

8.3 UI/UX Agent Contract

Input

Approved PRD
User Stories
Acceptance Criteria
Project Context

Responsibilities

- Tạo user flow.
- Tạo screen list.
- Tạo wireframe description.
- Tạo component inventory.
- Xác định loading/error/empty/success states.

Output

ux_spec.md
user_flow.md
wireframe_spec.md
component_inventory.md

Output quality rules

- Mỗi user story chính phải có user flow tương ứng.
- Mỗi screen phải có state phù hợp.

- UX spec phải đủ chi tiết để DEV Agent implement.
- Không được bỏ qua error states với critical flow.

Human gate

Gate 2: UX Approval

8.4 DEV Agent Contract

Input

Approved PRD
Approved UX Spec
Acceptance Criteria
Project Context

Responsibilities

- Tạo implementation plan.
- Xác định files/components/API affected.
- Tạo mock code diff hoặc real diff nếu tích hợp repo thật.
- Đánh giá risk.

Output

implementation_plan.md
mock_code_diff.md
changed_files.json
risk_assessment.md

Output quality rules

- Mỗi changed file phải có reason.
- Mỗi code diff phải map về acceptance criteria.
- Nếu động vào auth, DB, security, dependency thì risk = HIGH.
- Nếu risk = HIGH thì phải qua human approval trước khi QA.

Human gate

Gate 3: High-Risk Implementation Approval

8.5 QA Agent Contract

Input

PRD
Acceptance Criteria
UX Spec
Implementation Plan
Code Diff
Risk Assessment

Responsibilities

- Sinh test cases từ acceptance criteria.
- Tạo AC coverage matrix.
- Tạo QA report.
- Đánh dấu pass/fail/blocker.
- Đề xuất send back DEV nếu fail.

Output

test_cases.md
qa_report.md
ac_coverage_matrix.md

Output quality rules

- Mỗi acceptance criterion phải có ít nhất 1 test case.
- Test case phải có precondition, steps, expected result.
- QA report phải có pass/fail summary.
- Blocker bug phải chặn release.

Human gate

Gate 4: QA Review

9. Output Contract Layer

9.1 PRD Output Contract

Field	Required	Notes
Problem statement	Yes	Clear business/user problem
Target user	Yes	Who uses the feature
User stories	Yes	At least one story
Acceptance criteria	Yes	Measurable
Scope	Yes	What is included
Out-of-scope	Yes	What is excluded
Constraints	Yes	Technical/product constraints

9.2 UX Spec Output Contract

Field	Required	Notes
User flow	Yes	Main journey
Screen list	Yes	Required screens
Component inventory	Yes	Buttons, forms, alerts, etc.

Field	Required	Notes
Loading state	Yes	If async action exists
Error state	Yes	Required for critical flows
Success state	Yes	Required for task completion
Empty state	Optional	If relevant

9.3 DEV Output Contract

Field	Required	Notes
Implementation plan	Yes	Step-by-step plan
Changed files	Yes	File list
Code diff	Yes	Mock or real diff
AC mapping	Yes	Diff maps to AC
Risk assessment	Yes	Low/Medium/High

9.4 QA Output Contract

Field	Required	Notes
Test cases	Yes	Generated from AC
AC coverage matrix	Yes	Every AC covered
QA report	Yes	Pass/fail/blocker
Bug list	Yes	Can be empty
Release recommendation	Yes	Pass/Hold/Rework

10. Test Case Contract Layer

10.1 Purpose

Test Case Contract định nghĩa test case phải có format nào, kiểm tra acceptance criteria nào, và pass/fail dựa trên expected result nào.

10.2 Test Case schema

```
{
  "id": "TC-001",
  "source_ac": "AC-001",
  "title": "Google login button is visible",
  "type": "functional",
  "priority": "High",
  "precondition": "User is on login page",
  "steps": [
    "Open login page",
```

```

    "Check Google login button"
  ],
  "expected_result": "Google login button is visible and clickable",
  "actual_result": null,
  "status": "Not Run"
}

```

10.3 Required test types

Test Type	Purpose
Functional Test	Kiểm tra feature đúng requirement
Workflow Test	Kiểm tra PO → UX → DEV → QA chạy đúng
Agent Output Test	Kiểm tra artifact đủ field / đúng format
HITL Test	Kiểm tra approve/reject/request changes
Quality Gate Test	Kiểm tra gate chặn artifact lỗi
Audit Test	Kiểm tra log đủ actor/action/time

11. Quality Control Strategy

11.1 Đúng đề bài được đảm bảo bằng gì?

Đề bài yêu cầu	Cách chứng minh trong sản phẩm
End-to-end software development flow	Workflow board từ Feature Request đến Final Review
Team agent PO, UI/UX, DEV, QA	Mỗi phase có agent riêng và artifact riêng
Agent coordination	Artifact handoff qua database
Human control	Approval gates và audit trail
Demo trên software project	Mock project context + feature request + output artifacts
Product ready	UI/backend/state/approval chạy thật

11.2 Chất lượng tốt được đảm bảo bằng gì?

Quality dimension	Control mechanism
Requirement quality	PO output contract + human gate
UX quality	UX output contract + human gate
Implementation quality	DEV output contract + risk assessment
QA quality	AC coverage matrix + QA report
Process quality	Audit trail + state machine
Release quality	Final review packet + release decision gate

12. System Architecture

12.1 Logical architecture

flowchart LR

FE[Frontend

Workflow Board + Artifact Viewer]

API[Backend API]

WF[Workflow Engine

State Machine + Orchestrator]

AG[Agent Layer

PO / UX / DEV / QA]

QC[Quality Gate Service

Approval + Risk Rules]

ART[Artifact Service]

AUD[Audit Service]

DB[(SQLite / PostgreSQL)]

FE --> API

API --> WF

WF --> AG

WF --> QC

AG --> ART

QC --> AUD

ART --> DB

AUD --> DB

WF --> DB

12.2 Main backend services

Workflow Orchestrator

- Creates workflow run
- Moves state from phase to phase
- Handles reject/rework loop

Agent Runner

- Runs PO/UX/DEV/QA mock or LLM-backed agents
- Validates agent output against output contract

Artifact Service

- Saves artifacts
- Retrieves artifacts for UI
- Maintains artifact versioning

Approval Service

- Stores human decisions
- Applies gate transition rules

Audit Log Service

- Records all actions
- Provides timeline for mentor/user

Quality Gate Service

- Validates output contract
- Checks risk rules
- Checks AC coverage

12.3 Recommended stack

Frontend:

- React / Next.js
- Workflow board
- Markdown artifact viewer
- Approval modal
- Audit timeline

Backend:

- FastAPI or Node.js
- REST API
- Workflow state machine

Database:

- SQLite for MVP
- PostgreSQL if team wants production-like setup

Agent Layer:

- Mock first
- LLM optional with fallback

13. UI Scope

13.1 Dashboard

Shows:

- Feature list
- Current phase
- Status
- Progress
- Last updated

13.2 Workflow Detail Page

Shows:

PO Analysis → UX Design → Development → QA Review → Final Review

Each phase shows:

- Agent name
- Status
- Artifact count
- Human gate state
- Timestamp

13.3 Artifact Viewer

Left panel:

PRD
User Stories
UX Spec
Implementation Plan
Code Diff
QA Report
Final Packet

Right panel:

Markdown / JSON viewer

13.4 Approval Panel

Actions:

Approve
Reject
Request Changes
Add Comment

13.5 Audit Timeline

Example:

10:01 Feature created
10:02 IntentGate validated request
10:03 PO Agent generated PRD
10:05 Human approved PRD
10:07 UX Agent generated UX spec
10:10 Human requested UX changes
10:12 UX Agent revised UX spec
10:15 DEV Agent generated implementation plan
10:17 QA Agent generated test cases
10:20 Human approved final release

14. MVP Scope

14.1 Must-have

- Web app chạy được
- Feature request intake
- Workflow board
- 4 agents: PO, UI/UX, DEV, QA
- Data Contract definitions
- Agent Contract definitions
- Output Contract validation
- Artifact viewer
- Human approval gates
- Reject/rework loop
- QA test case generation
- AC coverage matrix
- Audit trail
- Final review packet

14.2 Should-have

- SQLite persistence
- Risk classification
- Agent self-review loop
- Seed demo project
- Error/loading/empty states
- Export final report

14.3 Nice-to-have

- Real LLM integration
 - GitHub PR integration
 - Real code diff from repo
 - Deploy preview
 - Slack/Teams notification
 - Session replay
-

15. Implementation Plan

Week 1 — Foundation

- Chốt workflow states
- Chốt data contracts
- Chốt agent contracts
- Chốt output contracts
- Setup frontend/backend skeleton
- Setup database schema

Week 2 — Core workflow

- Create feature request API/UI
- Implement workflow run
- Implement PO Agent mock
- Implement artifact store
- Implement Human Gate 1

Week 3 — Full agent chain

- Add UI/UX Agent
- Add DEV Agent
- Add QA Agent
- Implement artifact viewer
- Run full PO → UX → DEV → QA flow

Week 4 — HITL + Quality gates

- Implement approve/reject/request changes
- Implement reject/rework loop
- Implement output contract validation
- Implement risk gate
- Implement AC coverage check

Week 5 — Product polish

- Workflow board polish
- Audit timeline
- Final review packet
- Seed demo scenario
- Loading/error/empty states

Week 6 — Demo hardening

- Fix bugs
- Prepare demo script
- Add backup mock data
- Prepare report/slides
- Rehearse product demo

16. Demo Scenario

16.1 Sample feature request

Feature: Google Login

Description: Add Google login to the task management app so users can sign in faster.

Priority: High

Target user: End user

Constraints:

- Must not break existing email login
- Must support login failure state
- Must redirect to dashboard after successful login

16.2 Expected agent outputs

PO Agent:

- PRD
- User stories
- Acceptance criteria
- Scope/out-of-scope

UI/UX Agent:

- Login user flow
- Login screen states
- Google button component spec
- Error message behavior

DEV Agent:

- Implementation plan
- Changed files
- Mock diff
- Risk assessment: HIGH because auth flow is touched

QA Agent:

- Functional test cases
- AC coverage matrix
- QA report
- Recommendation: Pass/Hold/Rework

17. Risks and Mitigation

Risk	Impact	Mitigation
LLM output unstable	Demo fail	Mock-first agent outputs + fallback
Scope too wide	Cannot finish in 6 weeks	Focus on workflow platform, not real autonomous coding
HITL too annoying	Bad UX	Approve only quality gates
Test cases too generic	Low quality	Generate from AC + Test Case Contract
Mentor expects product	Report not enough	Build web app with real

Risk	Impact	Mitigation
Agent coordination unclear	Fails core requirement	workflow and approval Use artifact handoff + state machine